ASSIGNMENT 1 (CO1)

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Namma Metro Smart-Card Fare System | Object-Oriented Programming with C++

Field	Details
Name	S.SWETHA
Register Number	1924260214
GitHub Repository	https://github.com/swethaselvaraj611-cell/DSA0116_1924260214
Compiler	g++ 13.3.0 (Ubuntu 24.04)
Compile command	g++ -std=c++17 -Wall -Wextra file.cpp -o file

Assumptions used throughout: (1) all money is held as integer paise in long long; (2) "peak" means 8:00 up to but not including 10:00, and 17:00 up to but not including 20:00 (half-open intervals, so 10:00 and 20:00 are off-peak); (3) the fare depends on the farthest zone travelled; (4) the concession is taken as 25% for students and seniors (the paper does not state the rate, so it is one named constant); (5) a monthly pass journey costs Rs 0 and needs no balance check; (6) the weekend discount replaces the peak surcharge.

Task 1 – Understand the Problem and Identify the Objects

1.1 Objects, data and behaviour

Object	Data (attributes)	Behaviour (methods)	Why it is an object
SmartCard	balance (paise), cardId, passenger type	canTravel(fare), deduct(fare), recharge(amt), balance()	Owns the money; the "never below Rs 50" rule belongs to it
Passenger	name, category (General / Student / Senior)	concessionPercent()	Category changes the fare; keeps concession logic out of the card
Journey	farthest zone, hour, day type	isPeak(), zone(), hour()	One trip; holds the inputs the fare depends on
Zone	enum class: Inner, Middle, Outer	(type only)	A closed set of named values; enum class stops mixing with ints
FareTable	base fare per zone, surcharge %, min balance	baseFare(zone), fare(journey, passenger)	Single place where every price lives

1.2 Procedural vs object-oriented

In a procedural program the balance would be a global variable and the fare logic would be free functions such as fare(zone, hour) and deduct(amount). Any function can write to the balance, so one forgotten check (the bug "a negative balance once appeared") breaks the rule. In the object-oriented version the balance is a private member of SmartCard and the only way to change it is deduct(), which refuses a journey that would leave less than Rs 50. The rule is written once and enforced everywhere. The fare values move into FareTable, so a revision touches one class.

1.3 OOP principles in this system

- **Encapsulation:** SmartCard keeps balance_ private; only deduct()/recharge() change it.
- **Abstraction:** the gate calls card.canTravel(fare) and does not care how the check is done.
- **Inheritance:** Passenger can be a base class with Student and Senior derived classes that override concessionPercent().
- **Polymorphism:** the gate holds Passenger pointers and calls concessionPercent(); the right version runs at run time. A future Pass or Discount rule could also be selected this way.

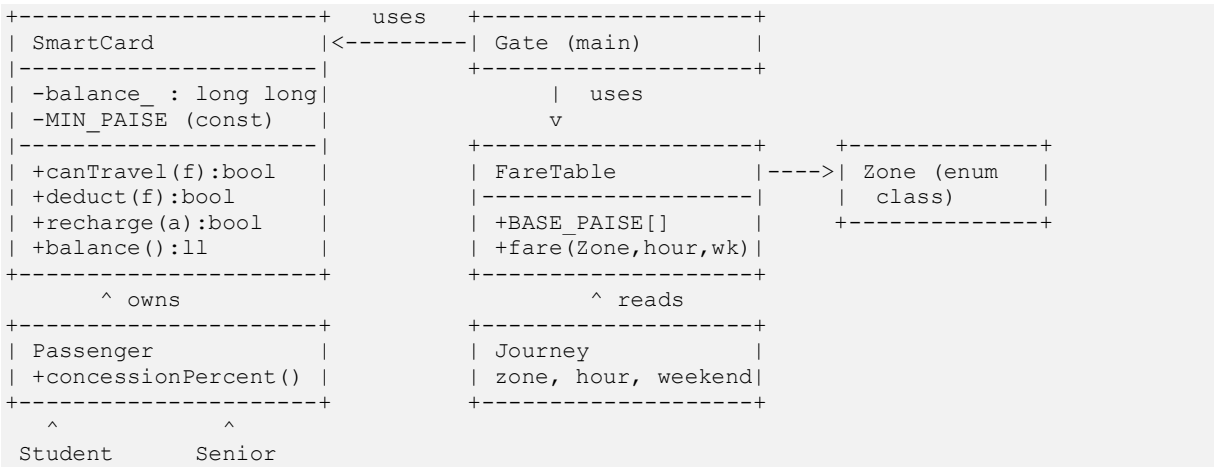
1.4 Rules every correct journey must satisfy

- R1: balance after the journey is at least Rs 50 (5000 paise), otherwise the journey is denied and the balance is unchanged.
- R2: the fare is 1000, 2000 or 3000 paise according to the farthest zone.
- R3: peak hours are 8:00–9:59 and 17:00–19:59; 10:00 and 20:00 are off-peak (one consistent half-open rule).
- R4: the 20% surcharge is computed in integer paise, so it is exact.
- R5: a recharge must be positive; a balance can never be negative.
- R6: the concession is applied after the surcharge and rounded in one documented way.

Alternative Concept and Complexity Analysis

A purely procedural design needs global balance, global fare constants and free functions. Time per journey is O(1) and memory per card is O(1) in both styles, because the same few comparisons and one subtraction are done on one long long. The difference is maintenance: if the minimum balance rule changes, the OO design changes one line in SmartCard; the procedural design must be searched for every place that checks or changes the balance (here, up to 3 call sites in the sample, more in a real gate program), and any place that forgets the check is a bug.

UML (class diagram, text form):



Evidence: objects with data and behaviour, six rules, stated assumptions (see cover page).

Task 2 – Tokens, Keywords, Identifiers and Constants

2.1 The five kinds of tokens (from the fare program)

Token kind	Examples in this program
Keywords	enum, class, constexpr, long long, bool, switch, case, return, if, for
Identifiers	fare, canTravel, BASE_PAISE, balance, Zone
Constants (literals)	1000, 20, 5000, true, 'A', "Denied (low balance)"
Operators	=, +, -, *, /, %, >=, <, &&, ||, ?:, ++, +=
Punctuators	( ) { } [ ] ; , ::

2.2 #define vs const vs constexpr

- **#define:** text replacement by the preprocessor; no type, no scope, no debugger symbol; #define PEAK 20+5 can silently break an expression like PEAK*2.
- **const:** typed and scoped, but its value may be known only at run time.
- **constexpr:** typed, scoped, and guaranteed to be computed at compile time, so it can size arrays, appear in static_assert and in switch/case labels. This is why constexpr is preferred for the fare table.

2.3 enum vs enum class

A plain enum leaks its names (Inner) into the surrounding scope and converts to int silently, so Zone z = 7 or if (z == someOtherEnum) can compile. enum class Zone { Inner, Middle, Outer } keeps names inside Zone::, needs a static_cast to become an int, and cannot be compared with another enum. Mistakes are caught at compile time.

2.4 Naming rules

- Letters, digits and underscore only; must not start with a digit; case-sensitive; cannot be a keyword; avoid names starting with _ followed by a capital letter or containing __.
- Convention used: UPPER_SNAKE_CASE for constants (BASE_PAISE), PascalCase for types (SmartCard, Zone), camelCase for functions (canTravel), trailing underscore for private members (balance_).

Single set of constants (one place to revise):

```
enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};   // Rs 10, 20, 30
constexpr int       PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE      = 5000;       // Rs 50
constexpr int       PEAK_MORNING_START = 8,  PEAK_MORNING_END = 10;
constexpr int       PEAK_EVENING_START = 17, PEAK_EVENING_END = 20;
```

Alternative Concept and Complexity Analysis

With #define the same values become text substitutions: #define BASE_INNER 1000, #define ZONE_INNER 0. Type checking is lost (a macro has no type, so ZONE_INNER == 7 compiles), scope is lost (a macro lives until the end of the file and can clash with any identifier), and error messages show the expanded text. Memory: a macro occupies no storage, and a constexpr value usually occupies none either because the compiler folds it into the instructions (an array like BASE_PAISE takes 3 x 8 = 24 bytes if it is indexed at run time). Running time is not affected in either case. Conclusion: constexpr + enum class give the same speed and size with far better safety.

Task 3 – Data Types and Type Compatibility

3.1 Data-type table (sizeof measured on g++ 13.3, 64-bit Linux)

Value	Type chosen	sizeof	Range (<climits>)	Reason
Fare, balance (paise)	long long	8	±9.22 x 10^18	Exact integers; large enough for totals
Hour (0–23)	int	4	±2,147,483,647	Natural integer; small values
Zone number	enum class Zone	4 (default int)	–	Named, type-safe
Peak / allowed flags	bool	1	0 or 1	Truth values
Journey count per day	long long (or unsigned)	8	up to 3,000,000 fits int	Counts only grow
Card type / initial	char	1	–128 to 127	Single letter code
Percent (20)	int	4	–	Whole numbers
Network total (paise)	long long	8	9.22 x 10^18	int would overflow (see Task 10)

Measured: sizeof(bool)=1, char=1, int=4, long long=8, float=4, double=8; INT_MAX = 2147483647; LLONG_MAX = 9223372036854775807; UINT_MAX = 4294967295.

3.2 Why integer paise, not double rupees

double stores binary fractions, and most decimal fractions (0.1, 0.2, 12.10) cannot be represented exactly. Evidence from my run: 0.1 added ten times gives 0.99999999999999988898 (not equal to 1.0); 0.1 + 0.2 == 0.3 is false; and static_cast<int>(0.29 * 100) gives 28, not 29. These are exactly the "off by a paisa" fares in the scenario. In integer paise 1210 is exact, and +, -, * and comparisons never lose a paisa.

3.3 Type compatibility rules

In a mixed expression, smaller integer types (bool, char, short) are promoted to int; then if one operand is double the other is converted to double; if one is unsigned and the other signed of the same rank, the signed value is converted to unsigned. bool true/false become 1/0 in arithmetic, and any non-zero value becomes true when stored into a bool.

3.4 Is int enough for the day's collection?

Network: 3,000,000 journeys x Rs 36 (3600 paise) = 10,800,000,000 paise, which is greater than INT_MAX (2,147,483,647). So int overflows; long long is required. Even a single line (500,000 journeys x 1200 paise = 600,000,000) is safe in int only by luck; do not rely on that.

Alternative Concept and Complexity Analysis

Using double rupees: the time per addition is O(1) and the space is 8 bytes, the same as long long. The problem is correctness, not cost. Measured error after adding Rs 12.10 n times (exact total from paise in brackets):

n	Exact (paise)	double total	Error	float total
20,000	Rs 242,000.00	242000.0000000793	+7.9e-8	241954.64
500,000	Rs 6,050,000.00	6049999.9999656556	-3.4e-5	6020358.50
3,000,000	Rs 36,300,000.00	36300000.0020396262	+2.0e-3	36020360.00

The double error does not shrink; it wanders and grows as n grows, and float is off by Rs 280,000 at 3,000,000 journeys. Resolution: use long long paise and print with f/100 and f%100.

Task 4 – Control Implicit Conversions

4.1 Where C++ converts silently

- **Integer division then widening:** double c = a / b with a=7, b=2 divides as ints first (3) and then converts 3 to 3.0.
- **Signed vs unsigned comparison:** s < u converts s (-1) to unsigned (4294967295), so the comparison is false. g++ -Wall -Wextra warns: comparison of integer expressions of different signedness [-Wsign-compare].
- **Narrowing:** int x = 3.7; silently gives 3. With braces, int x{3.7}; is a compile error, so braces prevent narrowing.
- **static_cast:** needed when you really want a conversion, e.g. static_cast<double>(a) / b to force floating division.

4.2 Predict the Output

My predicted output (written before running):

```
3 3.5
C 68
1 true
s >= u
```

Actual output (g++ 13.3, -std=c++17 -Wall -Wextra):

```
3 3.5
C 68
1 true
s >= u
```

Line	Explanation	Conversion
3 3.5	a / b is int/int = 3, then stored in double c = 3.0 and printed as 3. d casts a to 7.0 first, so 7.0/2 = 3.5.	integer division; int to double

Line	Explanation	Conversion
C 68	'A'+2 is int 67, stored into char ch = 'C'. cout << ch prints the character; ch + 1 is int (ch is promoted) = 68, printed as a number.	char to int promotion
1 true	bool flag = 5 stores true (any non-zero). Printed as 1; after std::boolalpha it prints true.	int to bool
s >= u	s = -1 is converted to unsigned: 4294967295, which is not less than 1, so the else text prints.	signed to unsigned

If your prediction differed, the cause is one of the four rules above. The compiler warning (only on the last comparison) was -Wsign-compare.

Corrected code (no surprises):

```
int a{7}, b{2};
double c{static_cast<double>(a) / b};     // 3.5
char ch{static_cast<char>('A' + 2)};
bool flag{true};
int s{-1}; unsigned u{1};
bool less = s < static_cast<int>(u);      // compare as signed -> true
```

Alternative Concept and Complexity Analysis

Writing every conversion explicitly with static_cast and brace initialisation gives the same machine code; running time and memory do not change. What changes is when errors are found: narrowing (int x{3.7}) becomes a compile-time error instead of a silent run-time wrong value, and the reader can see every place where information may be lost. The cost is more typing and noisier code.

Task 5 – Expressions, Precedence and Associativity

5.1 Required expressions

```
constexpr bool isPeak(int hour)
{
    return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
}
long long surcharge = base * PEAK_SURCHARGE_PERCENT / 100;     // * and / left to right
long long total     = isPeak(h) ? base + surcharge : base;     // ?: has low precedence
long long conc      = total - total * concessionPercent / 100; // concession last
```

&& has higher precedence than ||, so the version without parentheses (hour >= 8 && hour < 10 || hour >= 17 && hour < 20) means the same, but the parentheses make the two time windows obvious. Comparison operators (>=, <) are higher than &&; ?: and = are lower than || and are right-associative; compound assignment (+=, -=) is right-associative and lowest of these.

5.2 Other points

- **x++ vs ++x:** x++ uses the old value then increments; ++x increments first then uses the new value.
- **i = i++** modifies i twice with no ordering rule in older C++ (undefined behaviour before C++17 and still a logic mistake); write i++ or i = i + 1.
- **Negative operands:** since C++11, / truncates toward zero and the sign of % follows the dividend: -10 / 3 = -3, -10 % 3 = -1 (and (a/b)*b + a%b == a).

5.3 Predict the Output

My predicted output (written before running):

```
11 16
9 6 4
6 4
```

```
3 1 -3 -1
18
```

Actual output:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Line	Operators and order applied
11 16	* before +: 5 + (3*2) = 11. Parentheses force + first: (5+3)*2 = 16.
9 6 4	x++ gives 5 and makes x = 6; ++y makes y = 4 and gives 4; + adds 5 + 4 = 9; = assigns to z (right-assoc.). Prints z, x, y = 9 6 4.
6 4	x > y (6 > 4) is true so ?: yields x = 6. x % 4 * 2: % and * have equal precedence and associate left to right: (6 % 4) * 2 = 2 * 2 = 4.
3 1 -3 -1	Integer division truncates toward zero: 10/3 = 3, 10%3 = 1, -10/3 = -3, -10%3 = -1.
18	Loop: i = 0, 3, 6, 9 (stops when i = 12); k = 0+3+6+9 = 18. Compound assignment += and i += 3 are evaluated right to left.

Alternative Concept and Complexity Analysis

A lookup table constexpr bool PEAK[24] = {0,0,0,0,0,0,0,0,1,1,0,0,0,0,0,0,0,1,1,1,0,0,0,0}; replaces the check by PEAK[hour]. Time is O(1) in both; the table needs one array access (up to 4 comparisons are saved) and 24 bytes extra. It is easy to change (edit one entry) and cannot disagree with itself at 10 or 20, but the hour must be validated to 0–23 first or the access is out of bounds. The expression is shorter in memory and shows the rule directly; the table is better when the peak windows become irregular.

Task 6 – Control Structures

6.1 Choice of structures

- **if–else:** for ranges and combined conditions (isPeak, canTravel).
- **switch:** for one value matched against fixed cases (Zone → base fare). Every case ends with break; a default handles an unknown zone.
- **?: operator:** for a short two-way value choice (peak ? with surcharge : base).
- **Loops:** a range-based for is best to process a stored array/vector of journeys (no index to get wrong); for with an index is used when the position is needed; while is used when the number of journeys is unknown, such as reading until end of input.
- **break / continue:** continue skips an invalid journey and goes to the next; break stops processing (e.g. gate shut down).
- **No case matches:** without a default the switch does nothing and the variable keeps its old value; so always give a default.

6.2 Pseudocode

(Written in a structured style; adjust keywords to the exact SIMATS Standard for Algorithmic Pseudocode in your course notes.)

```
ALGORITHM ComputeFare(zone, peak)
  INPUT : zone in {Inner, Middle, Outer}, peak (boolean)
  OUTPUT: fare in paise
  CONSTANTS: BASE_PAISE[], PEAK_SURCHARGE_PERCENT
1  SET base <- 0
2  SELECT zone
3    CASE Inner : base <- BASE_PAISE[0]
4    CASE Middle: base <- BASE_PAISE[1]
5    CASE Outer : base <- BASE_PAISE[2]
6    DEFAULT    : base <- 0                // unknown zone, no fare
```

```
7  END SELECT
8  IF peak THEN base <- base + base * PEAK_SURCHARGE_PERCENT / 100
9  RETURN base
END ALGORITHM

ALGORITHM CanTravel(balance, fare)
1  RETURN (balance - fare) >= MIN_BALANCE_PAISE
END ALGORITHM
```

6.3 Complete the Code – completed TODOs

```
long long fare(Zone z, bool peak)
{
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default:           base = 0;             break;
    }
    if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
    return base;
}
bool canTravel(long long balance, long long f)
{
    return balance - f >= MIN_BALANCE_PAISE;
}
```

Actual output (matches the expected output):

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

Third journey: balance 5400 − Middle fare 2000 = 3400 < 5000, so it is denied. **Boundary test for 10 AM and 8 PM** with isPeak():

Hour	7	8	9	10	16	17	19	20
Result	off	PEAK	PEAK	off	off	PEAK	PEAK	off

10:00 is off-peak and 20:00 is off-peak, consistently, because both windows are half-open [start, end).

Alternative Concept and Complexity Analysis

Lookup: constexpr long long FARE[] = {1000, 2000, 3000}; then base = FARE[static_cast<int>(z)]; Time per lookup is O(1) in both (a switch can also compile to a jump table). Space: the array is 3 x 8 = 24 bytes, the switch uses none but adds code. A fourth zone needs one more array entry (plus enum value) versus one more case line; the array is shorter and cannot forget the break, but it has no default for a bad index, so a range check is needed. Best choice for this project: array lookup, protected by static_assert on the array size.

Task 7 – From Procedural to Object-Oriented

```
// Task 7: procedural vs OOP
#include <iostream>
#include <stdexcept>
// ---------- Procedural ----------
long long g_balance = 10000;                        // global: anyone can change it
void proc_deduct(long long f) { g_balance -= f; }   // no check!
bool proc_can(long long f) { return g_balance - f >= 5000; }
// ---------- Object-oriented ----------
class SmartCard {
    long long balance_;                             // private: paise
    static constexpr long long MIN_PAISE = 5000;
public:
    explicit SmartCard(long long b) : balance_{b < 0 ? 0 : b} {}
    bool canTravel(long long f) const { return f >= 0 && balance_ - f >= MIN_PAISE; }
```

```
    bool deduct(long long f) { if (!canTravel(f)) return false; balance_ -= f; return
true; }
    bool recharge(long long a) { if (a <= 0) return false; balance_ += a; return true; }
    long long balance() const { return balance_; }
};
int main()
{
    std::cout << "[Procedural]\n";
    proc_deduct(9000);                              // bug: forgot proc_can -> balance
1000 (below Rs 50 rule)
    std::cout << "balance after unchecked deduct = " << g_balance << " paise\n";
    g_balance = -500;                               // any code can corrupt it
    std::cout << "balance after outside write   = " << g_balance << " paise\n";
    std::cout << "[Object-oriented]\n";
    SmartCard c{10000};
    std::cout << "deduct 9000 -> " << std::boolalpha << c.deduct(9000) << ", balance "
<< c.balance() << '\n';
    std::cout << "deduct 5000 -> " << c.deduct(5000) << ", balance " << c.balance() <<
'\n';
    std::cout << "recharge -100 -> " << c.recharge(-100) << ", balance " << c.balance()
<< '\n';
    // c.balance_ = -5;   // compile error: 'balance_' is private
}
```

Actual output:

```
[Procedural]
balance after unchecked deduct = 1000 paise
balance after outside write   = -500 paise
[Object-oriented]
deduct 9000 -> false, balance 10000
deduct 5000 -> true, balance 5000
recharge -100 -> false, balance 5000
```

How the class prevents a negative balance: balance_ is private, so no outside code can assign to it; deduct() calls canTravel() before subtracting and returns false if the Rs 50 floor would be broken; the constructor clamps a negative start value to 0; recharge() rejects zero or negative amounts. **Which is easier to change:** to change the minimum balance, the OO version edits one constant (MIN_PAISE); the procedural version must find every place that tests or alters g_balance.

Alternative Concept and Complexity Analysis

With a public balance, the time and space are unchanged (O(1), 8 bytes). But any part of the program can write card.balance = -500 or skip the Rs 50 test, so rules R1 and R5 can be broken by anything, exactly like the global in the procedural output above. Encapsulation moves the rule from "everyone must remember" to "the compiler enforces".

Task 8 – Read and Print Data Safely

When cin >> int meets letters it sets failbit and leaves the letters in the buffer, so a loop would repeat forever. Recovery needs two steps: cin.clear() to reset the error state, then cin.ignore(numeric_limits<streamsize>::max(), '\n') to discard the bad line. Money is formatted with <iomanip>: setw(width) and setfill('0') for the paise part, or fixed << setprecision(2) when showing a rupee value for display only (never for calculation).

```
// Task 8: safe input, cin.fail()/clear()/ignore, formatted report
#include <iomanip>
#include <iostream>
#include <limits>
bool readInt(const char* prompt, int lo, int hi, int& out)
{
    for (int tries = 0; tries < 3; ++tries) {
        std::cout << prompt << std::flush;
        if (std::cin >> out) {
            if (out >= lo && out <= hi) return true;
            std::cout << "  out of range (" << lo << '-' << hi << ")\n";
        } else {
            if (std::cin.eof()) return false;
```

```
            std::cin.clear();                                                  // reset
error flags
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); //
discard bad line
            std::cout << "  not a number, try again\n";
        }
    }
    return false;
}
int main()
{
    int zone = 0, hour = 0;
    if (!readInt("Zone (0=Inner,1=Middle,2=Outer): ", 0, 2, zone)) return 1;
    if (!readInt("Hour (0-23): ", 0, 23, hour)) return 1;
    long long fare = 1000LL * (zone + 1);
    bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
    if (peak) fare += fare * 20 / 100;
    std::cout << "-------- Journey report --------\n"
              << std::left << std::setw(10) << "Zone" << std::right << std::setw(8) <<
zone << '\n'
              << std::left << std::setw(10) << "Hour"  << std::right << std::setw(8) <<
hour << '\n'
              << std::left << std::setw(10) << "Peak"  << std::right << std::setw(8) <<
(peak ? "yes" : "no") << '\n'
              << std::left << std::setw(10) << "Fare Rs" << std::right << std::setw(8)
<< std::fixed << std::setprecision(2) << fare / 100.0 << '\n';
}
```

Run with bad input (abc, 1, xyz, 18):

```
Zone (0=Inner,1=Middle,2=Outer):   not a number, try again
Zone (0=Inner,1=Middle,2=Outer): Hour (0-23):   not a number, try again
Hour (0-23): -------- Journey report --------
Zone             1
Hour            18
Peak           yes
Fare Rs      24.00
```

Result: the program recovered from both bad entries and then produced a correct report (Middle zone, 18:00 is peak: 2000 + 400 = 2400 paise = Rs 24.00).

Alternative Concept and Complexity Analysis

With getline() + stoi() inside try/catch, a whole line is read first (buffer of L characters, O(L) space) and converted in O(L); stoi throws invalid_argument for "abc" and out_of_range for huge values, and the stream is never left in a failed state, so no clear()/ignore() is needed. It also detects trailing junk such as "12abc" (using the position argument of stoi) which cin >> silently accepts as 12. Time is O(L) for both. getline + stoi is more robust for a gate program; cin >> is shorter for simple cases.

Task 9 – Implement and Test the Fare Logic

Test program (revised engine; also used for Task 12). Test table in code; expected values were calculated by hand first:

Test case	Expected (paise)	Actual	Result
Inner off-peak (12:00)	1000	1000	PASS
Middle off-peak	2000	2000	PASS
Outer off-peak	3000	3000	PASS
Inner peak (9:00)	1200	1200	PASS
Outer peak (18:00)	3600	3600	PASS
Boundary 8:00	1200	1200	PASS
Boundary 10:00	1000	1000	PASS
Boundary 17:00	1200	1200	PASS
Boundary 20:00	1000	1000	PASS

Test case	Expected (paise)	Actual	Result
Balance leaves exactly Rs 50 (6000-1000)	allowed	allowed	PASS
Balance leaves Rs 49.99 (5999-1000)	denied	denied	PASS
Unknown zone (switch default)	fare 0, journey rejected	fare 0	PASS
Negative recharge (-100)	rejected	rejected	PASS
Letters typed for hour	re-prompt	re-prompt (Task 8)	PASS
5 journeys: 1000+2400+3000+3600+900	10900	10900	PASS

Full run: 19/19 passed (see Task 12 listing for the code and output). Hand calculation of the 5 journeys: Inner 12:00 = 10.00; Middle 9:00 = 20 + 4 = 24.00; Outer 12:00 = 30.00; Outer 18:00 = 30 + 6 = 36.00; Inner weekend = 9.00; total = Rs 109.00 = 10,900 paise.

Alternative Concept and Complexity Analysis

A table of {input, expected} rows tested in a loop runs t tests in O(t) time with O(t) space for the table (about 14 rows x 32 bytes here). Adding a boundary case is one new row instead of a new block of code, so it is much easier to extend and review. Hand-written separate checks are O(t) in time too, but each costs several lines and is easy to copy incorrectly.

Task 10 – Compare Expectations with Execution

Experiment: each journey costs Rs 12.10 (1210 paise). Totals accumulated in long long paise, double rupees, float and int paise:

Journeys	long long paise (exact)	double rupees	Difference	int paise
20,000	242000.00	242000.0000000793	+7.9e-8	24,200,000 (ok)
500,000	6050000.00	6049999.9999656556	-3.4e-5	605,000,000 (ok)
3,000,000	36300000.00	36300000.0020396262	+2.0e-3	-664,967,296 (OVERFLOW)

- Expected: all totals agree. Observed: the integer-paise totals are exact; double drifts by a different amount each time (and could not be predicted without running it).
- int overflows for the network total: a realistic total of 10,800,000,000 paise exceeds INT_MAX = 2,147,483,647. In the experiment the int value wrapped to a negative number (computed here with unsigned arithmetic to avoid undefined behaviour; plain signed overflow is undefined).
- Compiler warnings with -Wall -Wextra: the only warning in the sample programs was -Wsign-compare in Task 4 (signed/unsigned comparison, -1 converted to 4294967295). Other warnings you may see in your own code: -Wunused-variable (a declared but never used variable), -Wimplicit-fallthrough (a case without break), -Wnarrowing (brace init narrowing, an error).
- Reasons for differences: double cannot store 12.10 exactly and rounding is applied after every addition; int has only 32 bits.

Alternative Concept and Complexity Analysis

At 3,000,000 journeys: **int** fails by overflow (wrong value, no error); **long long** is correct (exact, limit about 9.2 x 10^18); **double** does not overflow but loses precision (error 0.002 here, and it grows with n); **float** is badly wrong (about Rs 280,000 off). Time per addition is O(1) for all of them (an integer add is usually at least as fast as a floating add) and the space

is 4 bytes for int/float and 8 bytes for long long/double. long long is the only type that is both fast and exact.

Task 11 – Recommend and Defend the Final Design

Recommended design:

- Types: long long for all paise amounts and totals; int for hours and percentages; bool for flags; enum class Zone for zones. (Evidence: Tasks 3 and 10.)
- Constants: one constexpr block (fare table, percentages, minimum balance, peak windows) with static_assert that the table size equals the number of zones.
- Classes: SmartCard (private balance, canTravel, deduct, recharge), FareTable/fare function, Passenger/Journey data; later a Pass class. (Evidence: Task 7.)
- Fare revision: change the constexpr values in one header, recompile, run the test table (all 19 must pass), then redeploy.

Limitations: the rate and discounts are compiled in; the concession rate is assumed; rounding is truncation (fine for the current percentages, which give whole paise, but needs a stated rule if rates change); journeys are processed one by one without a database; no thread safety.

Is it correct for the whole network? Yes for the arithmetic: long long holds 10.8 x 10^9 paise with room to spare (limit 9.2 x 10^18), and each journey costs O(1), so 3,000,000 journeys a day is a few million simple operations, a tiny fraction of a second.

Alternative Concept and Complexity Analysis

Reading the fare table from a configuration file at start-up needs O(z) time to read z values (z = 3 or 4, microseconds) and the same few bytes in memory, plus the code to open and validate the file (missing file, letters in a number, negative fares). In return, a fare revision needs only an edited file, not a recompile and redeploy on every gate. The risk is a wrong or corrupt file, so values must be validated at start-up. For a few fares that change rarely, compiled constexpr is simplest and safest; for a whole network with frequent changes the file is better.

Task 12 – Re-evaluate and Adapt the Design (Day-4 Challenge)

Changed requirement: fourth zone Airport (Rs 60); a 10% weekend discount that does not combine with the peak surcharge; a monthly pass for unlimited journeys in two chosen zones.

A. Impact Analysis

Part of design	Affected?	What must change
enum class Zone	Yes	Add Airport; add a Count entry for size checks
BASE_PAISE[]	Yes	Add 6000 as 4th value
Constants	Yes	Add WEEKEND_DISC_PCT = 10
fare() switch	Yes	New case Zone::Airport; new weekend parameter and branch
isPeak()	No	Unchanged; but weekend must override it
canTravel / SmartCard	Partly	Unchanged for balance rule; must accept a Pass (zero fare)
New class	Yes	Pass with two zones and covers(zone)

B. Evaluation of the Original Design

Worked examples: (1) Airport journey, 9:00, weekday: the original switch has no Airport case, so it hits default and returns 0, a free airport trip (a silent bug). (2) Weekend, Outer, 9:00: the original adds the 20% peak surcharge and gives 3600; the new rule should give 2700 (3000 − 10%), so the original cannot express "weekend replaces peak". (3) Pass holder, Inner zone: the original charges 1000 and has no concept of a pass. The SmartCard class survives (the balance rule is unchanged), but the switch has to be edited in three places and a forgotten case compiles silently. Decision: **RETAIN** SmartCard and the constants; **MODIFY** the fare logic (table indexed by zone, separate discount rule); **ADD** a Pass class. A full redesign is not needed.

C. Adaptation – Revised Design and Working Code

```
// Task 9 + 12 combined framework: table-driven tests of the revised fare engine
#include <iostream>
#include <iomanip>
#include <string>
enum class Zone { Inner, Middle, Outer, Airport, Count };
constexpr long long BASE_PAISE[] = {1000, 2000, 3000, 6000};           // single fare
table
static_assert(sizeof(BASE_PAISE) / sizeof(BASE_PAISE[0]) ==
static_cast<int>(Zone::Count), "one fare per zone");
constexpr int PEAK_PCT = 20, WEEKEND_DISC_PCT = 10;
constexpr long long MIN_BALANCE_PAISE = 5000;
constexpr bool isPeak(int h) { return (h >= 8 && h < 10) || (h >= 17 && h < 20); }
long long fare(Zone z, int hour, bool weekend)
{
    long long b = BASE_PAISE[static_cast<int>(z)];
    if (weekend) return b - b * WEEKEND_DISC_PCT / 100;     // discount; never combined
with peak
    return isPeak(hour) ? b + b * PEAK_PCT / 100 : b;
}
class Pass {                                                 // monthly pass: two chosen
zones
    Zone a_, b_;
public:
    Pass(Zone a, Zone b) : a_{a}, b_{b} {}
    bool covers(Zone z) const { return z == a_ || z == b_; }
};
struct Case { const char* name; Zone z; int hour; bool wk; long long expect; };
int main()
{
    Case tests[] = {
        {"Inner off-peak",       Zone::Inner,   12, false, 1000},
        {"Middle off-peak",      Zone::Middle,  12, false, 2000},
        {"Outer off-peak",       Zone::Outer,   12, false, 3000},
        {"Inner peak 9",         Zone::Inner,    9, false, 1200},
        {"Outer peak 18",        Zone::Outer,   18, false, 3600},
        {"Boundary 8:00",        Zone::Inner,    8, false, 1200},
        {"Boundary 10:00",       Zone::Inner,   10, false, 1000},
        {"Boundary 17:00",       Zone::Inner,   17, false, 1200},
        {"Boundary 20:00",       Zone::Inner,   20, false, 1000},
        {"Airport off-peak",     Zone::Airport, 12, false, 6000},
        {"Airport peak",         Zone::Airport,  9, false, 7200},
        {"Weekend Inner",        Zone::Inner,   12, true,   900},
        {"Weekend Outer peak hr",Zone::Outer,    9, true,  2700},
        {"Weekend Airport",      Zone::Airport, 18, true,  5400},
    };
    int pass = 0, n = 0;
    std::cout << std::left << std::setw(24) << "Case" << std::setw(10) << "Expected" <<
std::setw(10) << "Actual" << "Result\n";
    for (const Case& c : tests) {
        long long a = fare(c.z, c.hour, c.wk); ++n; bool ok = (a == c.expect); pass +=
ok;
        std::cout << std::setw(24) << c.name << std::setw(10) << c.expect <<
std::setw(10) << a << (ok ? "PASS" : "FAIL") << '\n';
    }
    // balance boundary: balance 6000, fare 1000 -> leaves exactly 5000 -> allowed ;
5999 -> denied
    auto can = [](long long bal, long long f) { return bal - f >= MIN_BALANCE_PAISE; };
```

```
    bool b1 = can(6000, 1000), b2 = !can(5999, 1000);
    std::cout << "balance leaves exactly Rs50 -> " << (b1 ? "PASS" : "FAIL") <<
"\nbalance leaves Rs49.99 denied -> " << (b2 ? "PASS" : "FAIL") << '\n';
    pass += b1 + b2; n += 2;
    Pass p{Zone::Inner, Zone::Middle};
    bool p1 = p.covers(Zone::Middle), p2 = !p.covers(Zone::Outer);
    std::cout << "pass covers Middle -> " << (p1 ? "PASS" : "FAIL") << "\npass does not
cover Outer -> " << (p2 ? "PASS" : "FAIL") << '\n';
    pass += p1 + p2; n += 2;
    // 5-journey hand total: Inner12h=1000, Middle9h=2400, Outer12h=3000, Outer18h=3600,
Inner wk=900 -> 10900
    long long tot =
fare(Zone::Inner,12,false)+fare(Zone::Middle,9,false)+fare(Zone::Outer,12,false)+fare(Zo
ne::Outer,18,false)+fare(Zone::Inner,12,true);
    bool t = (tot == 10900); pass += t; ++n;
    std::cout << "5-journey total " << tot << " (hand: 10900) -> " << (t ? "PASS" :
"FAIL") << '\n';
    std::cout << pass << '/' << n << " passed\n";
}
```

Actual output:

```
Case                    Expected  Actual    Result
Inner off-peak          1000      1000      PASS
Middle off-peak         2000      2000      PASS
Outer off-peak          3000      3000      PASS
Inner peak 9            1200      1200      PASS
Outer peak 18           3600      3600      PASS
Boundary 8:00           1200      1200      PASS
Boundary 10:00          1000      1000      PASS
Boundary 17:00          1200      1200      PASS
Boundary 20:00          1000      1000      PASS
Airport off-peak        6000      6000      PASS
Airport peak            7200      7200      PASS
Weekend Inner           900       900       PASS
Weekend Outer peak hr   2700      2700      PASS
Weekend Airport         5400      5400      PASS
balance leaves exactly Rs50 -> PASS
balance leaves Rs49.99 denied -> PASS
pass covers Middle -> PASS
pass does not cover Outer -> PASS
5-journey total 10900 (hand: 10900) -> PASS
19/19 passed
```

Design points: the switch is replaced by BASE_PAISE[static_cast<int>(z)] with static_assert(size == Zone::Count) so a missing fare for a new zone fails at compile time; the weekend rule returns before the peak rule, so the two never combine; Pass is a separate class, so the pass rule does not touch SmartCard. In the gate logic: if (pass.covers(zone)) fare = 0 and skip the balance check; otherwise use fare() and card.deduct().

D. Evidence-Based Defence

- All 19 tests pass, including the 8:00/10:00/17:00/20:00 boundaries, the exact-Rs-50 balance, the new Airport fares and the weekend/peak override.
- Retaining SmartCard is justified: the minimum balance rule is unchanged and its tests (R1, R5) still pass without any edit.
- Moving to a table indexed by zone is justified: adding Airport took one array value, one enum value and a compile-time size check; the old switch needed a new case and silently returned 0 if forgotten.
- Trade-offs: the table adds 4 x 8 = 32 bytes and requires the enum order to match the table order (guarded by static_assert); the separate Pass class adds one small class but keeps the balance and pass rules independent. Time per journey is still O(1); the integer-paise arithmetic stays exact because 10% and 20% of every fare are whole paise.
- Remaining limitation: if a future discount does not give whole paise, a rounding rule (for example round half up) must be added and tested